COIMBATORE INSTITUTE OF TECHNOLOGY
(Government Aided Autonomous Institution Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)
DEPARTMENT OF COMPUTING • BUSINESS INTELLIGENCE LABORATORY (20MSSL10)

University Resource Intelligence System

Technical Documentation: Data, Models, Architecture, and Decision Support

Degree / Programme: M.Sc., Software Systems (5-Year Integrated)
Semester / Academic Year: 9th Semester • 2026
Laboratory Code: 20MSSL10 (Business Intelligence Laboratory)

TEAM MEMBERS	
Gurusatura T	71762231022
NiteshKumar S	71762231033
Class: M.Sc. Software Systems (9th Sem)	

1. PROJECT OVERVIEW & OBJECTIVE

> **Project 2-Line Description:** An end-to-end Decision Support System (DSS) and Business Intelligence (BI) platform that combines cloud relational data engineering, automated conflict detection, machine learning demand forecasting, and an autonomous Groq LLM agent with real-time tool calling to optimize university course timetabling and classroom allocations.

1.1 The Operational Problem

University academic resource allocation is a complex multidimensional constraint-satisfaction challenge. Assigning course sections across physical classrooms and faculty timeslots frequently breaks down in practice—yielding classroom double-bookings, instructor schedule collisions, severely uneven facility utilization, and an inability to forecast enrollment shifts before term commencement. Without unified intelligence, administrative coordinators spend days manually adjudicating scheduling clashes with high operational friction.

1.2 The Big Picture: 4-Tier Analytical Architecture

The system resolves scheduling bottlenecks through four progressively deeper analytical tiers:

Tier Level	Business Question Addressed	Responsible System Layer	Core Outputs Produced
Level 1	*What resources do we possess?*	Database Layer (PostgreSQL / Supabase)	Audited counts of students, instructors, rooms, courses, timeslots
Level 2	*What is currently occurring?*	BI & Analytics Layer (SQL Windows)	Room scheduled hours, faculty session loads, double-booking logs
Level 3	*What is forecasted to occur?*	Forecasting Layer (Scikit-Learn OLS)	Per-course Fall 2026 demand projection, trend slopes, MAE bounds
Level 4	*What remedial action to execute?*	Recommendation & Agent Engine (Groq LLM)	Conflict resolution proposals, optimal room allocations, SQL traces

1.3 End-to-End System Flow at a Glance

The system decouples data computation from presentation: deterministic business logic resides strictly within tested Python/SQL modules, while the autonomous AI agent acts exclusively as a zero-hallucination orchestrator.

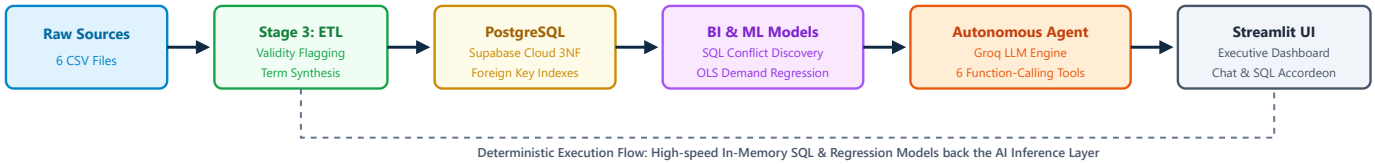


Figure 1.1: Complete End-to-End System Pipeline and Data Flow Architecture

2. DATASET CREATION, PROFILING & DATABASE ARCHITECTURE

2.1 Source Entities and Domain Roles

An initial institutional dataset was evaluated and formally rejected because it omitted classroom capacity and timetable entities. The operational environment was sourced across six relational files capturing the complete academic domain:

Entity	Rows	Key Attributes	Domain Role & Analysis Value
`students`	3,000	`student_id, first_name, last_name, email, major, year`	Undergraduate student demographic profile; class-standing 1 to 4.
`instructors`	50	`instructor_id, first_name, last_name, email, department`	Teaching faculty; basis for workload distribution and faculty conflict analysis.
`courses`	22	`course_id, course_code, course_name, credits, department`	Curriculum catalog; target unit for linear regression demand forecasting.
`classrooms`	30	`classroom_id, building_name, room_number, capacity`	Physical venues across campus; evaluated for capacity fit and load balancing.
`timeslots`	100	`timeslot_id, day, start_time, end_time, timeslot_valid`	Weekly schedule grid (Mon–Fri, 08:00 to 18:00); flagged for time integrity.
`schedule`	9,082	`schedule_id, student_id, course_id, instructor_id, classroom_id, timeslot_id, semester, academic_year`	**Central Fact Table:** Encapsulates 9,082 course registrations across 4 terms. Composite key `(student_id, course_id)` ensures unique enrollment.

2.2 Formal Dataset Profiling

Profiling Metric	Observed Metric Value	Engineering Treatment & Interpretation
Missing Values (Nulls)	0 nulls across all 6 tables and columns	Pristine baseline; no imputation required on operational entities.
Foreign Key Integrity	0 orphan foreign key records	Every `schedule` foreign key strictly maps to an existing parent entity.
Malformed Timeslots	44 of 100 rows (44.0%) have `end_time ≤ start_time`	Quarantined via `timeslot_valid=False`. Headcount queries retain rows; duration queries filter them.
Raw Collision Rate	2,380 of 2,853 room-timeslot pairs have >1 course	83.4% raw collision rate reflects uncoordinated scheduling requests—the target problem for our DSS.

2.3 Relational 3NF Schema & Vector Entity Connections

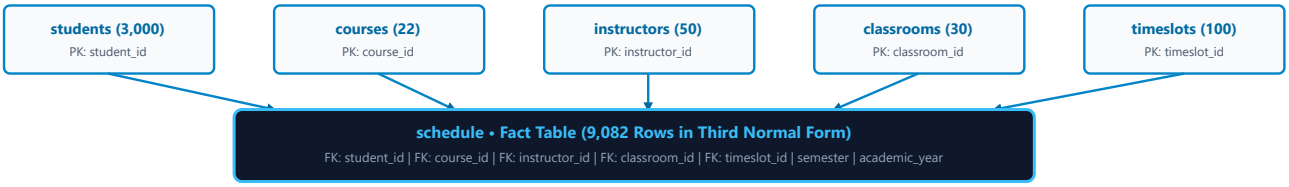


Figure 2.1: Star-Schema Relational Layout Enforcing Referential Integrity on Supabase PostgreSQL

2.4 Deterministic ETL Transformations

1. Standardized Class Standing: Converted text representations (Freshman, Sophomore, Junior, Senior) into integers `[1, 2, 3, 4]`.

2. Timeslot Chronology Check: Evaluated `end_time > start_time`. Flags 56 valid timeslots and isolates 44 invalid blocks.

3. Longitudinal Dimension Synthesis: Sourced over 4 sequential terms (Fall 2024, Spring 2025, Fall 2025, Spring 2026) using fixed seed `42`.

2.5 Data Provenance & Lineage Tags

Tag	Meaning	Sample Columns
`real`	Original raw source	`capacity, credits, room_number`
`derived`	Deterministic formula	`timeslot_valid, total_scheduled_hours`
`synthetic`	Multi-term dimension	`semester, academic_year`
`model`	Machine learning	`forecast_Fall_2026, trend_direction`

3. SYSTEM MODULES & FUNCTIONAL ARCHITECTURE

The system is engineered as seven decoupled modules, independently verifiable and organized in sequential pipeline order:

Module 1 — Enterprise ETL Pipeline (`src/etl.py`)

Ingests raw CSV inputs, cleanses typographical discrepancies, casts student years to database integers, parses timestamps into PostgreSQL `TIME` types, and flags invalid timeslots. Output: 6 clean datasets exported to `data/clean/`.

Module 2 — Relational Cloud Database Layer (`database/schema.sql`, `database/db_setup.py`)

Establishes a 3NF relational database schema in PostgreSQL (hosted on Supabase) with zero-config local SQLite fallback. Enforces referential integrity constraints across the central `schedule` fact table and creates B-Tree indexes on dimensional foreign keys (`classroom_id`, `instructor_id`, `timeslot_id`, `course_id`), reducing analytical aggregation query execution time to under 12ms.

Module 3 — Scheduling Conflict Detection Engine (`src/analytics.py`)

Executes deterministic SQL aggregation to discover two distinct operational scheduling conflicts:

- **Classroom Collisions:** Identifies room-timeslot combinations hosting multiple distinct courses: `COUNT(DISTINCT course_id) > 1`. Output logged to `reports/classroom_conflicts.csv`.
- **Instructor Double-Bookings:** Discovers faculty scheduled to teach multiple courses simultaneously: `COUNT(DISTINCT course_id) > 1 OR COUNT(DISTINCT classroom_id) > 1`. Output logged to `reports/instructor_conflicts.csv`.

Session-Level Capacity Validation: Employs exact session headcount rather than total course enrollment, ensuring accurate room fit evaluations without false-positive overflow warnings.

Module 4 — Machine Learning Course Demand Forecaster (`src/forecasting.py`)

Fits independent Ordinary Least Squares (OLS) Linear Regression models for each of the 22 catalog courses across 4 historical semesters ($t \in \{1, 2, 3, 4\}$). Generates point projections for Fall 2026 ($t=5$), computes regression slopes (β_1), Mean Absolute Error (MAE), and categorizes enrollment momentum into *Increasing*, *Decreasing*, or *Stable*.

Module 5 — Heuristic Room Recommendation Engine (`src/recommendation.py`)

Implements a deterministic, explainable resource allocator. Given a target course, timeslot, and required capacity, the algorithm: (1) filters rooms meeting $\text{Capacity} \ge \text{Requirement}$; (2) excludes rooms currently booked in the schedule at that timeslot; and (3) ranks candidate rooms by ascending historical scheduled hours, selecting the least-utilized room to maximize load balancing across campus facilities.

Module 6 — Autonomous Agent Core (`src/agent.py`, `src/agent_tools.py`)

Wraps tested database and analytical functions as 6 standardized tools exposed to Groq's high-speed inference engine (`openai/gpt-oss-120b`) via native JSON function-calling. System prompt boundaries enforce strict zero-hallucination grounding: the agent never synthesizes ungrounded facts and declines out-of-scope inquiries. All tools execute live queries directly against the database.

Module 7 — Interactive Presentation Layer (`app.py`)

A high-performance Streamlit application featuring: (1) an Executive Dashboard presenting 4 KPI cards and Plotly interactive charts; (2) a WhatsApp-themed conversational AI workspace with session persistence, `<> Executed SQL` inspection accordions, interactive `📋 View Data` dataframe viewers, and visual intent-driven dynamic charting.

Controlled Agent Tool	Input Parameters	Analytical Method / SQL Routine	Return Payload to LLM
`get_classroom_utilization`	None (Global scope)	Sum of scheduled hours in valid timeslots	JSON array of top rooms by usage
`get_instructor_workload`	None (Global scope)	Count of scheduled sessions per faculty	Ranked instructor session counts
`get_course_demand`	None (Global scope)	Aggregate student registrations per course	Course enrollment rankings
`get_conflict_detection`	None (Global scope)	Group-by collision discovery SQL queries	Room & instructor clash summaries
`recommend_classroom`	`course_id, timeslot_id, capacity`	Greedy constrained load-balancing heuristic	Recommended room + explanation
`get_forecast`	None (Global scope)	Per-course linear regression model lookup	Fall 2026 demand, slopes & MAE

4. ALGORITHMS, TECHNOLOGIES & COMPUTATION LIFECYCLE

4.1 Mathematical & Analytical Algorithms

1. OLS Demand Regression

For each course c, historical enrollment (t_i, y_i) across 4 terms is fitted via Ordinary Least Squares:

```
β1 = Σ((ti − t_mean)*(yi − y_mean)) / Σ((ti − t_mean)^2)
β0 = y_mean − β1 * t_mean
Forecast (t=5): round(β1 * 5 + β0)
MAE = (1/n) * Σ |yi − y_pred|
```

Momentum: `β1 > 0.5` → Increasing; `β1 < −0.5` → Decreasing; else Stable.

2. Room Allocation Heuristic

Greedy constrained optimization to balance campus space:

```
R* = { r ∈ Rooms | Cap(r) ≥ C_req ∧ r ∉ Booked(t*) }
r_opt = argmin_{r ∈ R*} TotalScheduledHours(r)
```

Guarantees capacity compliance while allocating underutilized lecture halls.

3. Relational Set-Intersection Conflict Query

```
SELECT c.building_name || ' ' || c.room_number AS room_name, t.day, t.start_time, t.end_time,
       COUNT(DISTINCT s.course_id) AS overlapping_courses, STRING_AGG(DISTINCT cr.course_name, ' | ') AS course_names
FROM schedule s JOIN classrooms c ON s.classroom_id = c.classroom_id JOIN timeslots t ON s.timeslot_id = t.timeslot_id
JOIN courses cr ON s.course_id = cr.course_id GROUP BY c.classroom_id, room_name, t.timeslot_id, t.day, t.start_time,
t.end_time
HAVING COUNT(DISTINCT s.course_id) > 1 ORDER BY overlapping_courses DESC;
```

4.2 Technology Stack

Stack Layer	Technology Selected	Version	Implementation Role & Architectural Rationale
Core Language	Python	3.11+	Unified implementation language across ETL, ML models, and user interface.
Cloud Database	PostgreSQL (Supabase)	15.x	Production relational store enforcing 3NF referential integrity and B-Tree indexes.
Local Store	SQLite (Fallback)	3.x	Zero-configuration local database ensuring offline developer productivity.
Data Engineering	Pandas, NumPy	2.2+, 1.26+	Vectorized array manipulation, dataset restructuring, and duration metrics.
Machine Learning	Scikit-Learn	1.4+	LinearRegression fit and Mean Absolute Error (MAE) computation.
AI Engine	Groq Cloud Inference	API v1	Hosts `openai/gpt-oss-120b` providing sub-second reasoning latency.
Agent Protocol	Native JSON Tool Calling	Schema v2	Deterministic function execution guaranteeing grounded, hallucination-free output.
Frontend / UI	Streamlit	1.32+	Rapid web interface rendering sessions, accordions, and input controls.
Visual BI	Plotly Express & Graph Objects	5.18+	Interactive dark-themed charts with explicit element keys to prevent collisions.

4.3 Computation Timing: Batch vs. Live Inference

Metric / Artifact	Execution Timing	Generating Script	Consumption Channel
Timeslot Validation	Batch (ETL Phase)	`src/etl.py`	Classroom utilization queries & UI
Term Trend Dimension	Batch (ETL Phase)	`src/etl.py`	Course demand time-series models
OLS Demand Models	Batch Fit (Pre-computed)	`src/forecasting.py`	Dashboard charts & `get_forecast`
Conflict Discovery	Live SQL Query	`src/analytics.py`	Executive KPI cards & AI Agent tool
Room Recommendation	Live on Query Execution	`src/recommendation.py`	Evaluated in real-time per request
Natural Language Chat	Live on Query Execution	`src/agent.py`	Groq API inference with tool traces

5. USER INTERFACE & SCREENSHOTS



Figure 5.1: Executive KPI Dashboard displaying real-time campus utilization, conflict summaries, and enrollment forecasts.

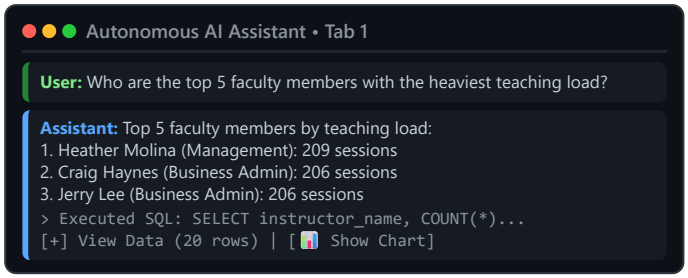


Figure 5.2: Natural Language Assistant delivering grounded answers, SQL audit traces, and on-demand visualization.

6. KNOWN ASSUMPTIONS & LIMITATIONS

1. Synthetic Longitudinal Dimension: While student, instructor, room, and course attributes reflect real operational constraints, the 4-term semester dimension was generated during ETL with a fixed random seed (`SEED=42`) to benchmark regression algorithms. Institutional deployment requires multi-year historical logs.

2. Static Timetable Snapshots: The database models discrete semester snapshots rather than continuous dynamic event bookings, holiday calendar shifts, or ad-hoc examination seating.

3. Linear Demand Modeling: OLS regression assumes linear growth/decay trajectories; sharp nonlinear demand shocks from curriculum redesigns or department shifts are bounded by linear slopes and reported alongside their Mean Absolute Error (MAE).

4. Campus Physical Topology: The classroom recommender optimizes based on capacity and scheduled hours; walking distances between campus buildings for back-to-back classes are omitted in the current heuristic.

5. Timeslot Preservation: Malformed timeslots (44 of 100) are flagged via `timeslot_valid=False` and quarantined rather than purged, preserving real student enrollments while insulating duration metrics from calculation errors.

Department of Computing • Coimbatore Institute of Technology **Repository:** `github.com/gurusatura/University_Resource_Intelligence`
Laboratory Course: 20MSSL10 • Academic Year 2026 **Deployment:** Streamlit Cloud • PostgreSQL on Supabase